# Introduction to Prompt Engineering and RAG (Retrieval Augmented Generation)

Dipendra Raj Bhatt

Zakipoint Health Nepal

30/03/2026

# Outline

# Prompt Engineering

- Prompt Engineering is the art and science of crafting inputs that guide LLMs to deliver accurate, structured, and reliable outputs.
- It works at the interface of human and machine, shaping the model's probabilistic output to achieve predictable and deterministic results.

# Why Prompt Engineering Exists

- LLMs are probabilistic, not deterministic, meaning their output can vary even for the same input.
- Prompt engineering mitigates common failures:
  - Hallucination: model invents facts to satisfy the prompt pattern
  - Format Drift: model returns text when a structured output (e.g., JSON) is expected
  - Context Amnesia: model forgets instructions buried in the middle of a long prompt
- Good prompt engineering structures inputs to ensure consistent, accurate, and predictable outputs.

# Temperature in LLMs: A Programmer's Perspective

- Temperature controls the randomness of LLM outputs.
- **High temperature (0.8–1.0+) → creative outputs**
  - Pros: brainstorming, naming variables, generating examples
  - Cons: unsafe for structured outputs

```
{
    "name": "Captain Stardust",
    "age": "Ageless (exists beyond time)"
}
```

- **Low temperature (0–0.2) → deterministic outputs**
  - Pros: predictable, safe, consistent formatting
  - Ideal for scripts, SQL queries, API responses, JSON

```
{
    "name": "John Doe",
    "age": 30
}
```

# Variants of Prompt Engineering

- Zero-shot/Few-shot
- Chain-of-thought
- Tree of thought
- Meta Prompting

# Few-shot / Zero-shot Prompting

- **Zero-shot prompting:** Give the model direct instructions without examples. It relies entirely on pre-trained knowledge.

```
Classify text into neutral, negative,or positive.
Text: I think the product is amazing!
```

- **Few-shot prompting:** Provide a few examples in the prompt to guide the model.

```
Example 1: "I love this movie!"      Positive
Example 2: "The service was terrible."      Negative
Classify: "The product is okay, nothing special."
```

# Chain-of-Thought (CoT) Prompting

- CoT prompting enables LLMs to solve complex problems by generating intermediate reasoning steps before giving a final answer.
- Instruct the model to "think step by step" to expand its reasoning process.
- It is especially useful for math, logic, and multi-step reasoning tasks.
- Example:

```
Question: If there are 12 apples and I buy 8 more,
    then give 5 to a friend, how many apples do I
   have?
Use a step-by-step approach to find the solution:
```

- Response:

```
Step 1: Start with 12 apples.
Step 2: Buy 8 more      12 + 8 = 20 apples.
Step 3: Give 5 to a friend      20 - 5 = 15 apples.
Answer: 15
```

# Tree of Thoughts (ToT) - Concept

- ToT is an advanced reasoning technique that explores multiple candidate paths, evaluates them, and selects the best solution.
- Think of it as a decision tree of possible thoughts rather than a single line of reasoning.
- Example Riddle:

```
Riddle: I speak without mouth & hear without ears.
I have nobody, but I come alive with wind.
```

# Tree of Thoughts (ToT) - Step-by-Step

- Step 1: Generate candidate thoughts
  - A: Animal? $\rightarrow$ No
  - B: Natural phenomenon? $\rightarrow$ Maybe
  - C: Man-made? $\rightarrow$ Possible
- Step 2: Expand reasoning
  - B $\rightarrow$ Wind gives life $\rightarrow$ Produces sound with wind
- Step 3: Evaluate  prune $\rightarrow$ Select the most promising path (B)
- Step 4: Follow the best path $\rightarrow$ Answer: Echo
- Final Answer: Echo

# Meta Prompting - Concept & Structure

- **Meta prompting**:Technique where the prompt guides how model should reason, focusing on structured thinking rather than just the task itself.
- Meta Prompt Example for Logic Puzzle Solving:

```
Problem Statement:
Problem: [logic puzzle question to be answered]
Solution Structure:
- Begin the response with: "Let's carefully analyze this step by step."
- Break down the logic and reasoning clearly, explaining each inference in order.
- Summarize the conclusion at the end in a clear statement:
  "Final Conclusion: [concise answer to the problem]."
- Optionally, provide a brief explanation confirming the answers validity.
```

**RAG(Retrieval Augmented Generation)**

# Historical Context

- **Memory-Augmented Neural Networks (MANNs):** Neural networks with an external "notebook" to store and recall information; they help remember facts over long tasks and make smarter decisions.
- **Knowledge-Enhanced Pre-training (e.g., ERNIE, K-BERT):** Models trained on text plus structured knowledge like knowledge graphs; improves understanding of entity relationships for accurate answers.
- **Neural IR + Generation Pipelines:** Early hybrid systems that first search for relevant info (IR) and then generate answers; like looking up sources before writing a report.
- **Retrieval-Augmented Generation (RAG):** Introduced by Lewis et al. (2020); jointly trains a retriever and generator for end-to-end learning on large-scale text corpora.

# Motivation: Bridging Generation and Retrieval

- Generative models alone struggle to dynamically incorporate new knowledge.
- Two key strands emerged:
  - **Closed-book Generation:** Relies only on pre-trained parameters; fluent but prone to hallucinations.
  - **Open-book Retrieval:** Retrieves relevant documents before answering; improves factual correctness but may lack fluent generation.
- **RAG:** Unifies these approaches by retrieving relevant context from external knowledge bases and conditioning generation on it.

# Retrieval-Augmented Generation (RAG)

- **Definition:** Retrieval-Augmented Generation (RAG) enables large language models (LLMs) to dynamically retrieve and incorporate new information from external sources, rather than relying solely on pre-trained knowledge. This allows for more accurate, up-to-date, and contextually grounded responses.
- **How RAG Works:**
  1. **Query Input:** A query $q$ is provided to the system.
  2. **Retrieval:** The retriever searches a knowledge base $D$ and selects the top-$k$ relevant documents $\{d_1, d_2, \ldots, d_k\}$.
  3. **Generation:** The generator produces the output $y$ conditioned on the query and retrieved documents:

$$y = G(q, \{d_1, d_2, \ldots, d_k\})$$

# Retrieval-Augmented Generation (RAG)

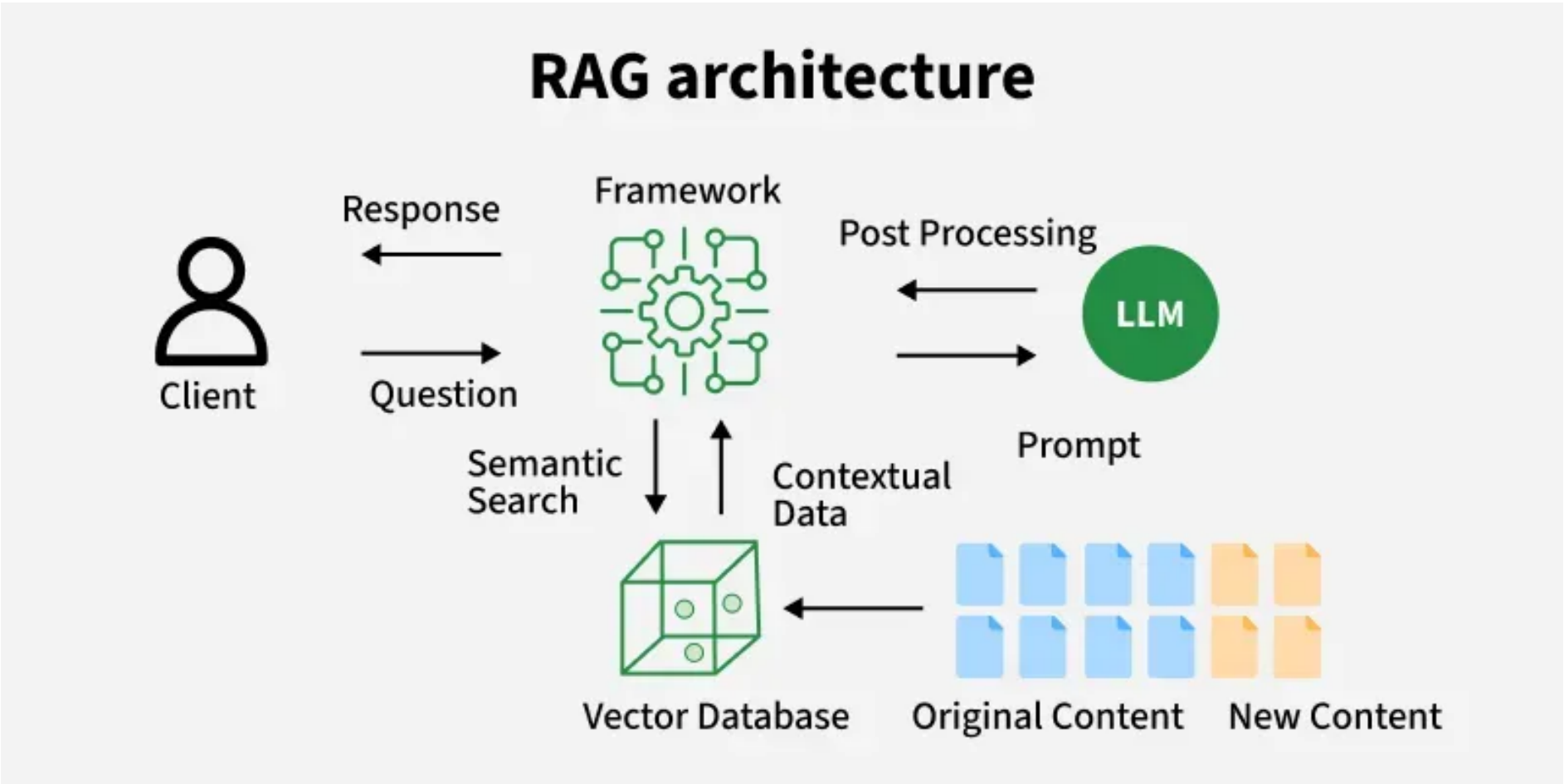


Figure: Architecture of RAG

# 1. Knowledge Base Construction

- Collect large source documents (PDFs, websites, reports)
- Split documents into smaller chunks
- Convert each chunk into vector embeddings
- Store embeddings in a vector database

# 1.Knowledge Base Construction

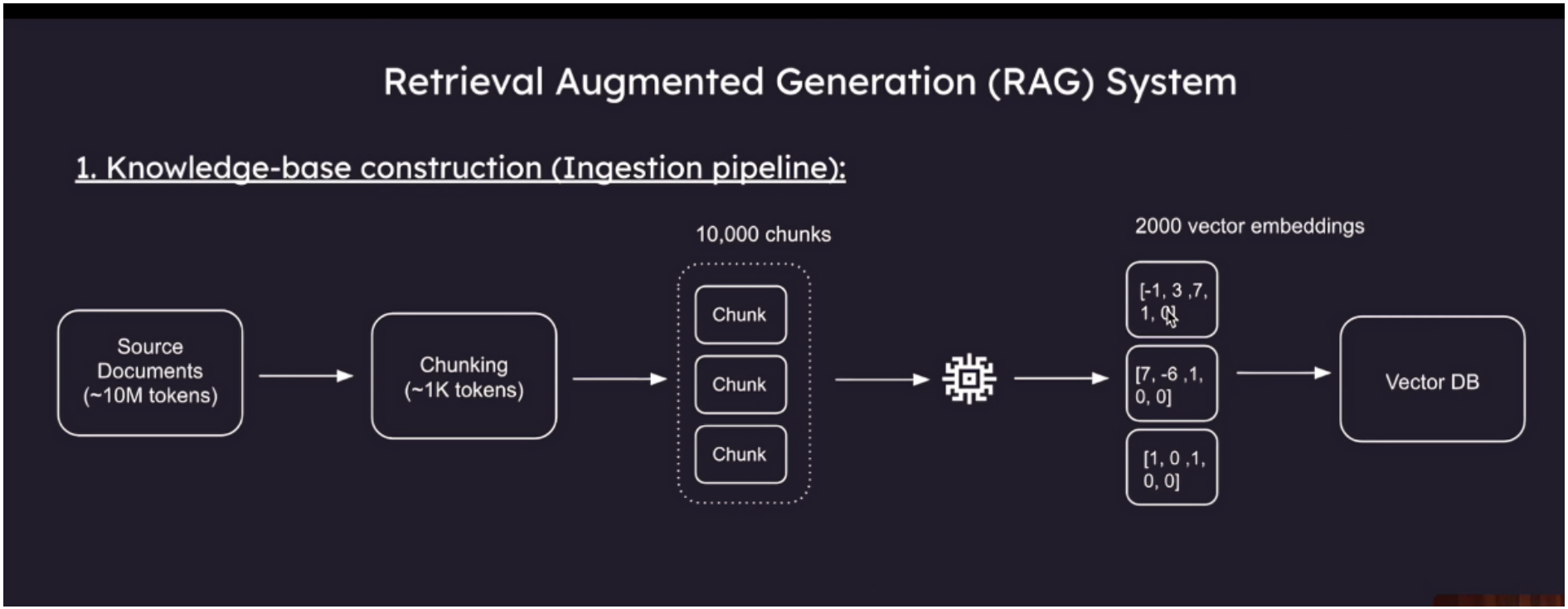


Figure: Knowledge Base Construction

# 2. Retrieval Pipeline

- User submits a query
- Query is converted into an embedding
- Retriever finds most relevant chunks
- Retrieved chunks are sent to the LLM
- LLM generates the final answer

# 2. Retrieval Pipeline

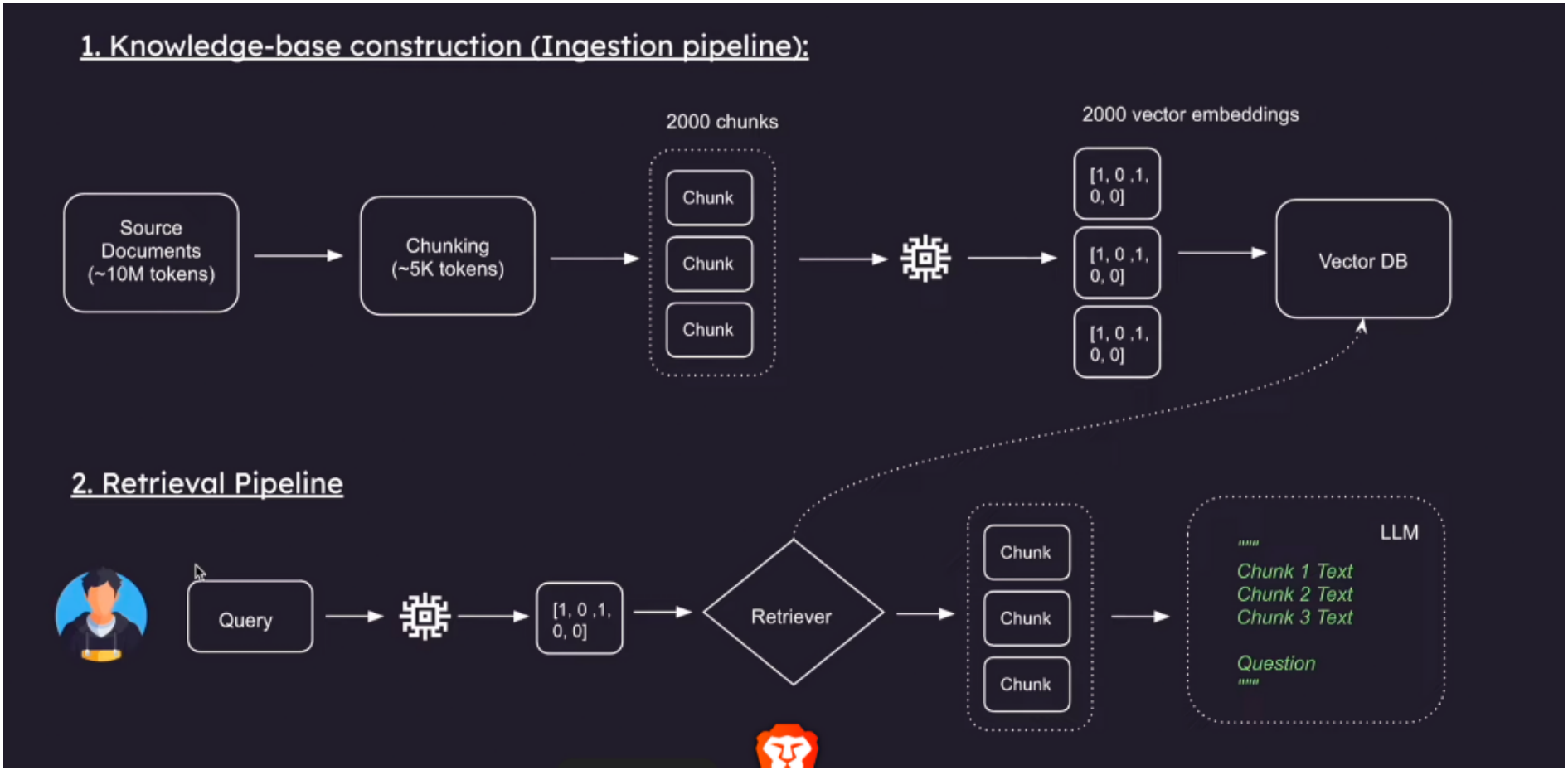


Figure: Ingestion  Retrieval Pipeline

# Variants of RAG

- Multimodal Retrieval-Augmented Generation (RAG)
- Graph Retrieval-Augmented Generation (RAG)
- Vectorless RAG (PageIndex)

# Multimodal Retrieval-Augmented Generation (RAG)

- Multimodal RAG extends traditional RAG to handle multiple data types such as **text, images, audio, and video**.
- The retriever fetches and combines different modalities (e.g., an image with its textual description or related audio).
- The generator processes and fuses these multimodal inputs to produce a **coherent, context-rich response**.
- **Example:** For a query like *"Show and describe the Eiffel Tower at night,"* the system retrieves an image, video clip, and text description, then generates a response that explains the landmark both visually and verbally.

# Multimodal Retrieval-Augmented Generation (RAG)

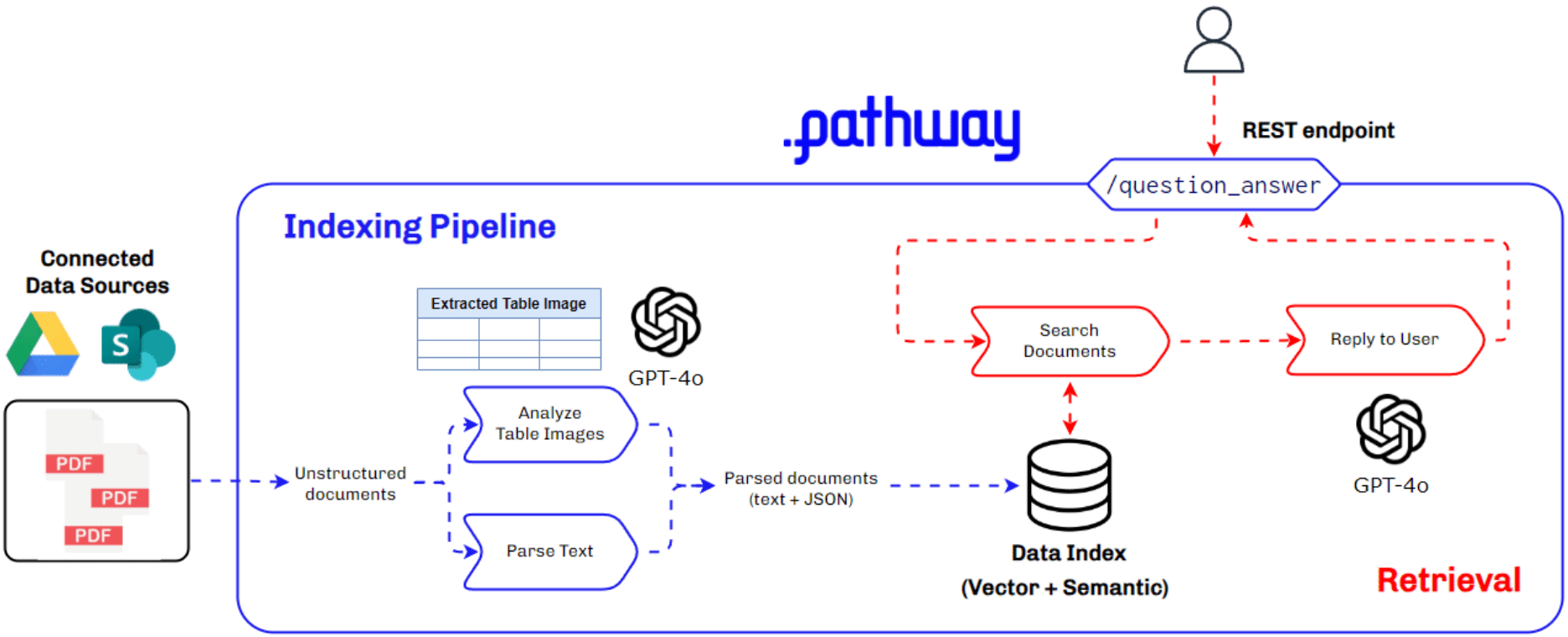


Figure: Multimodel RAG

# Multimodal Retrieval-Augmented Generation (RAG)

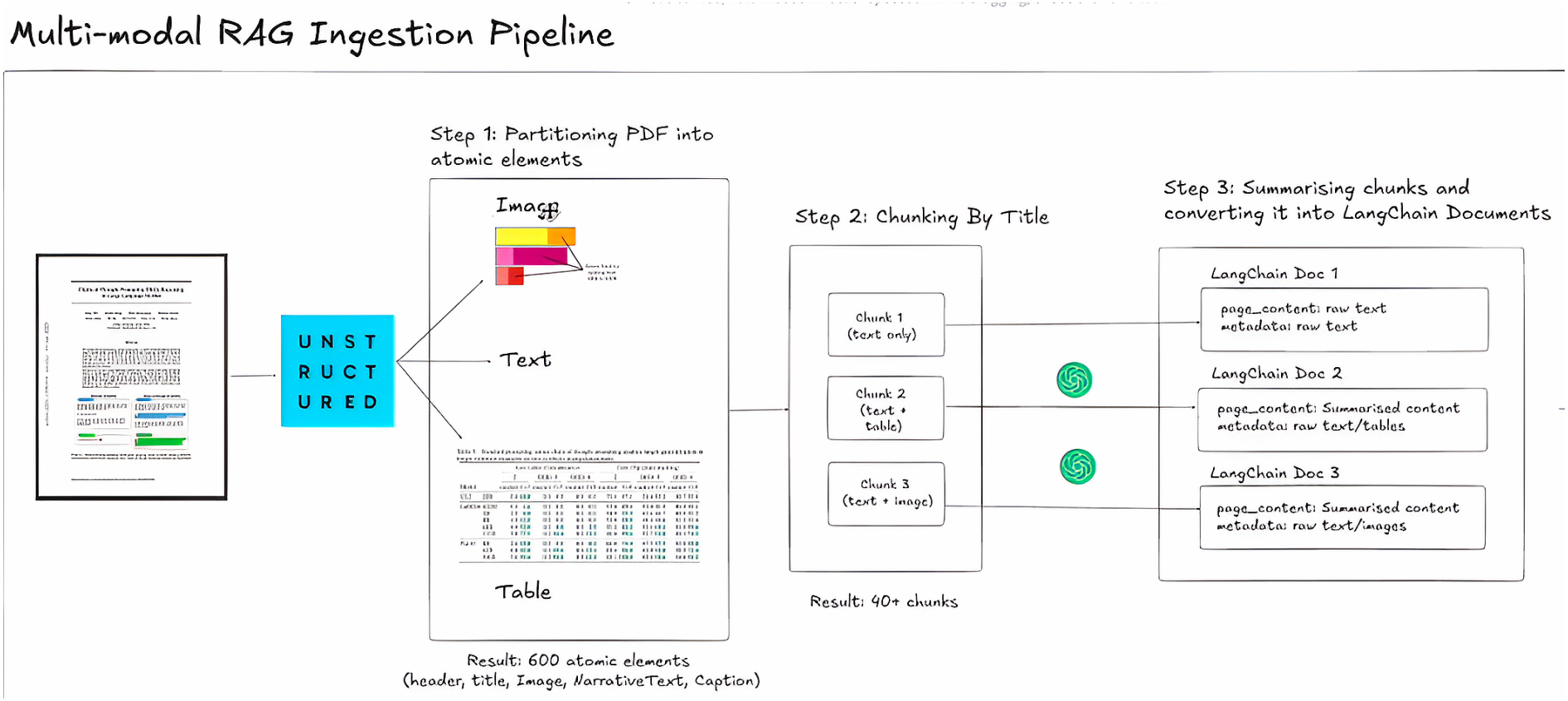


Figure: Ingestion Pipeline of RAG

# Graph Retrieval-Augmented Generation (RAG)

**Definition:**

- Graph RAG enhances retrieval by using a **knowledge graph** instead of (or along with) plain documents.

**Key Concepts:**

- Data is stored as **entities and relationships** (nodes and edges).
- Retrieval involves **traversing the graph** to find connected information.
- Enables **multi-hop reasoning** and better understanding of relationships.

**Example:**

- Finding how two entities are related (e.g., *company* $\rightarrow$ *founder* $\rightarrow$ *other companies*).

# Graph Retrieval-Augmented Generation (RAG)

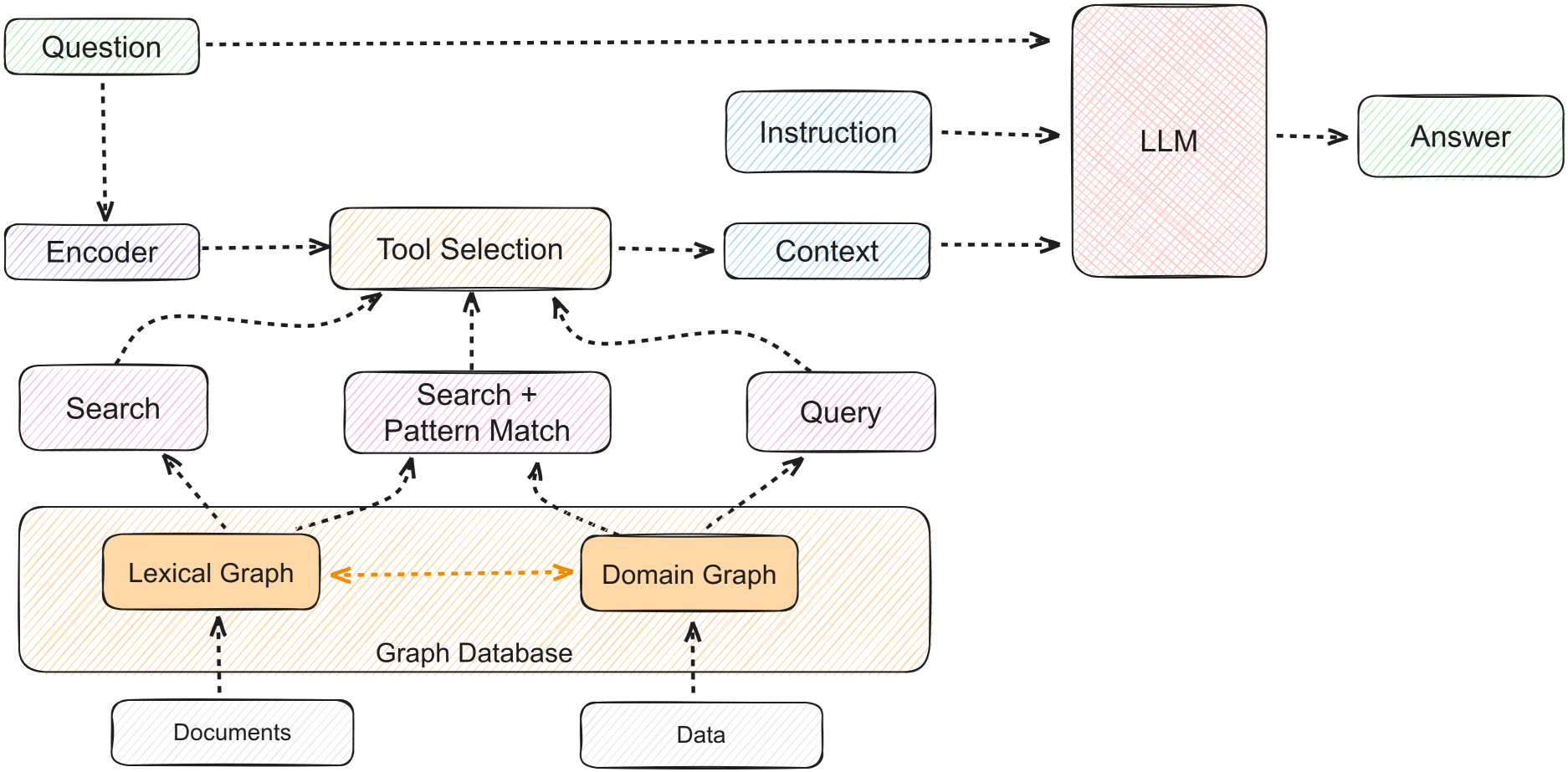


Figure: Graph RAG

# Vectorless RAG (PageIndex)

- Retrieval-augmented generation approach without using embeddings or vector databases.
- Organizes documents into natural sections or a tree-like index for step-by-step, reasoning-based retrieval.
- **Retrieves without vectors:** Uses document structure and keywords instead of embedding similarity.
- **Preserves context:** Maintains pages, headings, and sections to avoid losing meaning.
- **Reasoning-based retrieval:** Explores documents step by step using a tree-like index.
- **Transparent and interpretable:** Retrieval decisions are traceable and easy to understand.

# Page indexing RAG

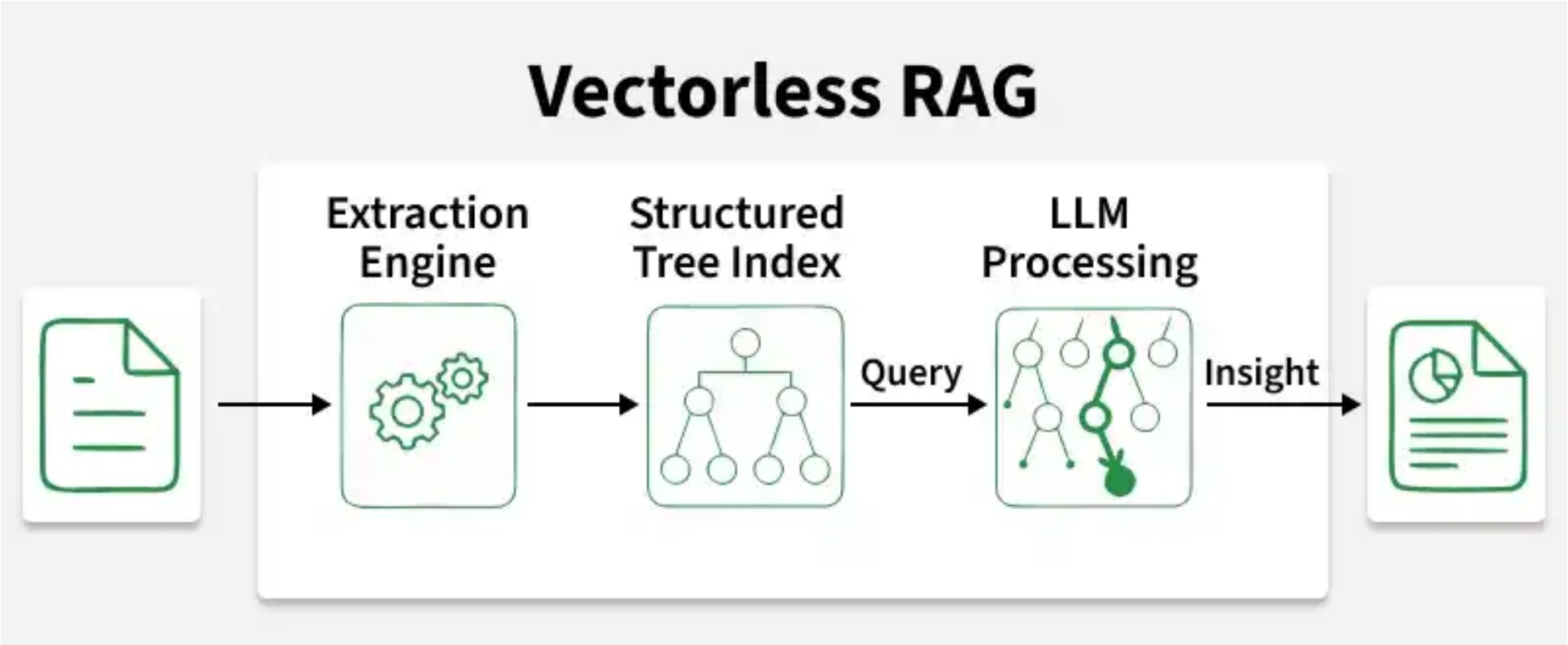


Figure: Abstract View of Page Indexing RAG

# Page indexing RAG

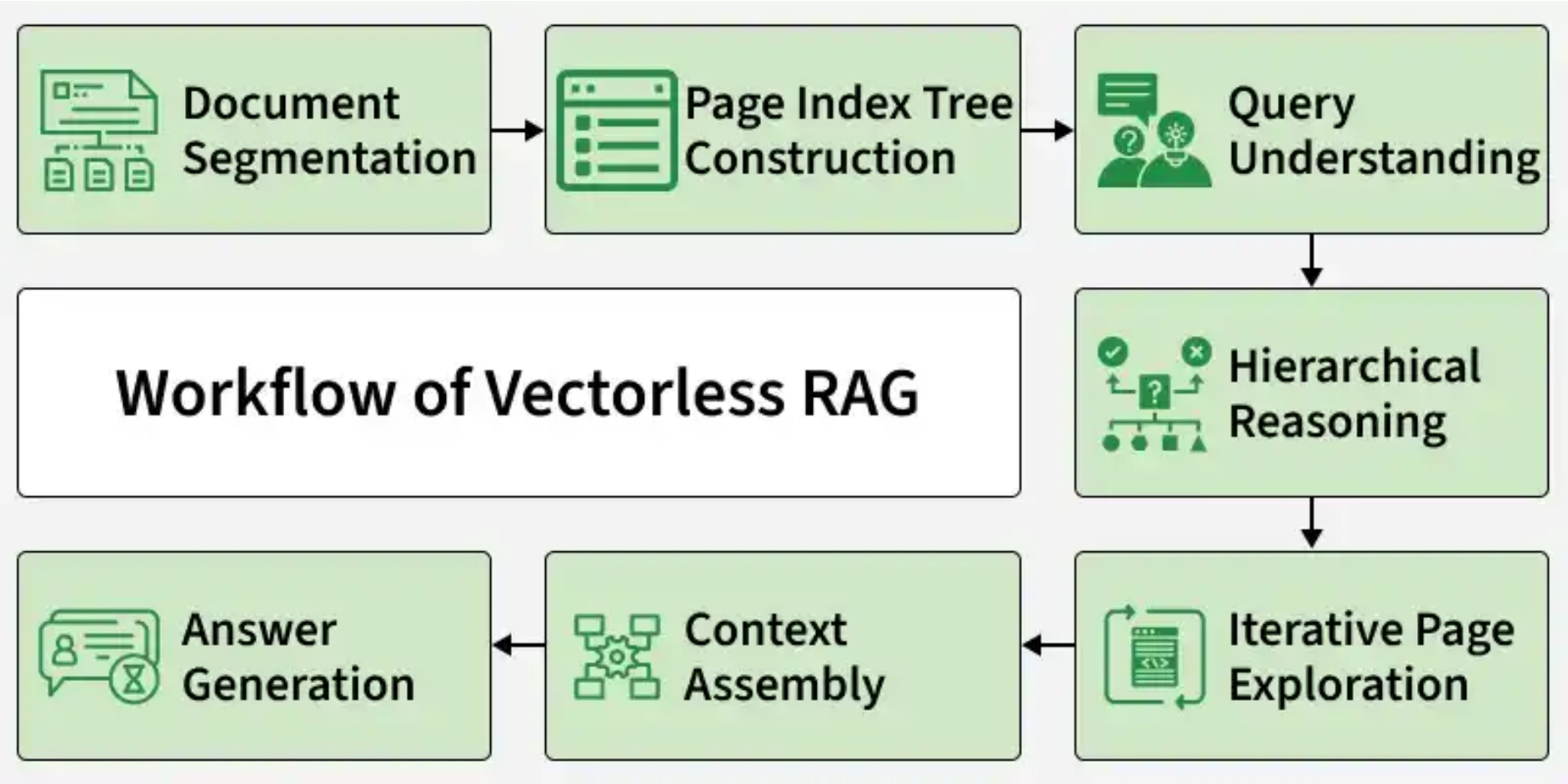


Figure: Work Flow of Page Indexing RAG

*Thank You!*